Zephyr Project

IT-Schulungen.com
Technology & Transformation

# Praxis 1: Erstes native_sim-Build

Geführtes Hands-on in Zweierteams

1.07

# Vom Sample zum geprüften Build

## Fünf Schritte mit sichtbarem Ergebnis

**01**

### Prüfen

hello_world und seine Dateien verstehen

**02**

### Bauen

Pristine Build für native_sim starten

**03**

### Beobachten

Konfiguration, Compiler und Linker erkennen

**04**

### Nachweisen

Zielvariante und Artefakte lokalisieren

**05**

### Vergleichen

Ergebnisse mit dem Partnerteam erklären

Ergebnis: zephyr.exe ist erzeugt; Zielvariante und mindestens drei Artefakte sind erklärt.

# Das Sample vor dem Build prüfen

ZWECK

## hello_world

Kleinster Build-Nachweis

Toolchain und Workspace prüfen

Konsolenausgabe vorbereiten

STRUKTUR

## Wichtige Dateien

src/main.c

CMakeLists.txt

optionale prj.conf

Sample-Metadaten

ERWARTUNG

## Sichtbares Ergebnis

Build endet erfolgreich

zephyr.exe entsteht

Ausgabe enthält Hello World

Vor dem Bauen kurz lesen: Welche Quelle wird übersetzt und welche Ausgabe wird erwartet?

# Pristine Build starten

TERMINAL

```
$ west build -p always -b native_sim
samples/hello_world

# nur bei Multilib-Fehler
$ west build -p always -b native_sim/native/64 \
  samples/hello_world
```

**01** **-p always**

Erzeugt den Build-Zustand für diese Übung kontrolliert neu.

**02** **64-Bit-Fallback**

Nur wechseln, wenn native_sim an fehlendem Multilib-Support scheitert.

Zuerst native_sim versuchen; die Zielvariante nicht ohne beobachteten Grund wechseln.

# Build-Phasen im Log erkennen

BUILD-LOG

```
-- Application: hello_world
-- Board: native_sim
-- Configuring done
-- Generating done
[1/…] Building C object
[…/…] Linking zephyr.exe
```

## 01 Konfigurieren

CMake, Kconfig und Devicetree legen den Build fest.

## 02 Übersetzen

Compiler erzeugt Objektdateien aus den ausgewählten Quellen.

## 03 Linken

Linker verbindet die Objekte zum ausführbaren zephyr.exe.

Markiert im Log je eine Zeile für Konfiguration, Übersetzung und Linken.

# Warnung oder tatsächlicher Fehler?

| | |
|---|---|
| **Warnung** | Build kann weiterlaufen; Meldung lesen und Ursache bewerten |
| **Fehler** | Build stoppt oder Zielartefakt fehlt; Ursache muss behoben werden |
| **Erste relevante Meldung** | Oft näher an der Ursache als die letzte sichtbare Fehlermeldung |
| **Folgefehler** | Spätere Meldungen können nur Konsequenzen derselben Ursache sein |
| **Teamregel** | Befehl, erste Fehlermeldung und Umgebung gemeinsam vergleichen |

Nicht nach Farben urteilen: Rückgabestatus, erstes Problem und fehlendes Artefakt entscheiden.

# Drei Artefakte und das Programm finden

**BUILD/ZEPHYR**

Alle Nachweise liegen im Build-Verzeichnis des erfolgreichen Builds.

| | |
|---|---|
| **zephyr.exe** | ausführbares Programm für den Linux-Host |
| **.config** | tatsächlich wirksame Kconfig-Einstellungen |
| **zephyr.dts** | zusammengeführte Hardwarebeschreibung |
| **zephyr.map** | Symbole und Speicherzuordnung des Linkers |
| **CMakeCache.txt** | gespeicherter Build-Kontext einschließlich Board-Auswahl |

# Ziel und Artefakte nachweisen

TERMINAL

```
$ test -x build/zephyr/zephyr.exe && echo OK
$ grep '^CONFIG_BOARD=' build/zephyr/.config
$ test -f build/zephyr/zephyr.dts && echo DTS_OK
```

## 01 Zielvariante

CONFIG_BOARD zeigt, für welches native_sim-Ziel gebaut wurde.

## 02 Artefakte

Ausführbarkeit und Dateinachweise bestätigen den erfolgreichen Build.

Der Nachweis kombiniert Rückgabestatus, Board-Konfiguration und vorhandene Build-Ergebnisse.

# Praxis 1 abgeschlossen

ERFOLG

## Build vollständig

Befehl erfolgreich

zephyr.exe vorhanden

Board-Ziel nachgewiesen

Drei Artefakte erklärt

PARTNERCHECK

## Beobachtungen teilen

Zielvariante vergleichen

Log-Phasen zeigen

Warnungen besprechen

Abweichungen notieren

NÄCHSTER SCHRITT

## Praxis 1.08

Programm starten

Ausgabe beobachten

Prozess beenden

Inkrementell neu bauen

Lernziel: Den Build, Zielvariante und Artefakte erklären können.